

MASTER PROMPT FOR ANTIGRAVITY

Last-Mile Delivery Tracker — Full-Stack Implementation

Copy everything below this line into Antigravity as a single prompt.

0. HOW TO USE THIS PROMPT

You (Antigravity) are being given one complete, self-contained specification for a production-ready **Last-Mile Delivery Tracker**. Build it end-to-end — setup, backend, frontend, database, tests, docs, deployment — without asking the requester what to do next. Where a decision was not specified by the source requirements, this document already makes that decision for you and labels it:

IMPLEMENTATION DECISION — a technical choice made on your behalf, with rationale. Follow it unless it becomes technically impossible, in which case document the deviation in the README.

Work in the phased order defined in Section 23. Do not skip phases. Do not stub functionality and call it done.

1. ROLE OF ANTIGRAVITY

You are acting simultaneously as:

Senior full-stack engineer — you own the implementation end to end.

Backend architect — you design and build a clean, layered Node.js/TypeScript API.

Frontend engineer — you build a production-quality React SPA.

Database architect — you design a normalized relational schema with correct constraints.

API designer — you produce a consistent, versioned, well-documented REST API.

Security engineer — you enforce authentication, authorization, and input safety everywhere.

QA engineer — you write and run unit, integration, and API tests before declaring anything done.

DevOps engineer — you configure environments, migrations, seeding, and deployment.

You own this project from `git init` through a live, hosted, demoable URL. You do not hand back partial work with "TODO" placeholders in business-critical paths.

2. PROJECT OVERVIEW

Project name: Last-Mile Delivery Tracker

Objective: A logistics operations platform where customers and admins create delivery orders with automatically calculated charges, orders are routed to the nearest available delivery agent, and customers are notified at every stage of the delivery journey — from pickup through delivery or failed-delivery rescheduling.

Problem being solved: Manual last-mile logistics pricing (zone lookup, weight-based billing, COD surcharges) and manual agent dispatch are slow and error-prone. This platform automates zone detection, billable-weight calculation, rate-card lookup, COD surcharge application, and nearest-agent assignment, while keeping a tamper-proof audit trail of every order's journey.

Main users:

Customer — registers, logs in, creates orders, previews charges, tracks deliveries, reschedules failed deliveries.

Delivery Agent — logs in, sets availability, receives assignments, updates delivery status.

Admin — configures zones/areas/rate cards/COD surcharges, manages agents, assigns/reassigns orders, overrides statuses, monitors all orders.

Core workflow: Order created → pickup/drop zone detected → volumetric weight calculated → billable weight = MAX(actual, volumetric) → correct rate card (B2B/B2C × intra/inter-zone) applied → COD surcharge added if applicable → charge shown for confirmation → order confirmed → agent assigned (manual or auto) → Picked Up → In Transit → Out for Delivery → Delivered **or** Failed → (if Failed) customer notified → customer reschedules → agent reassigned → delivery re-attempted → every transition logged immutably → customer notified by email on every status change.

Expected outcome: A deployed, working, testable application (backend API + frontend + database) with seed data demonstrating the full lifecycle, plus complete documentation and an 800-word system design write-up.

Scope:

In scope: customer/admin order creation, zone & rate-card driven pricing, manual + automatic agent assignment, full order status lifecycle, immutable tracking history, failed-delivery/reschedule/reassignment flow, email notifications (SMS abstraction with a free-tier or mock provider), admin configuration screens, role-based auth.

Explicitly excluded: payment gateway integration (payment type is recorded, not processed), live GPS map rendering/turn-by-turn navigation, multi-currency/multi-language support, native mobile apps (responsive web only), third-party courier integrations. These are out of scope unless the requester later asks for them.

3. FUNCTIONAL REQUIREMENTS

3.1 Customer

Feature	Input	Processing	Output	Validation	Error Handling
Registration	name, email, phone, password	hash password (bcrypt), create user with role=CUSTOMER	user record + auth tokens	email format, unique email, phone format, password ≥ 8 chars with 1 number	409 on duplicate email, 400 on invalid fields
Login	email, password	verify hash, issue JWT access + refresh token	tokens + user profile	required fields	401 invalid credentials, 429 after repeated failures (rate limited)
Order creation	pickup address, drop address, L/B/H (cm), actual weight (kg), order type (B2B/B2C), payment type (Prepaid/COD)	resolve zones, compute volumetric & billable weight, look up rate card, compute COD surcharge, compute total	draft order + charge breakdown (not yet persisted as CONFIRMED)	all dimension/weight fields > 0, addresses resolve to a configured area/zone, order type & payment type are valid enums	422 with field-level errors; 409 ZONE_NOT_CONFIGURED if address unmapped; 409 RATE_NOT_CONFIGURED if no matching rate card
Charge preview	same order payload	same calculation as above, no persistence	itemized breakdown: base charge, COD surcharge, total, billable weight used	same as above	same as above
Order confirmation	draft order id/payload	persist order as CREATED , write first tracking-history entry, trigger auto-assignment attempt (configurable)	confirmed order with order number	draft must not be stale (re-validate rates at confirmation time)	409 if rate card changed since preview — re-quote and ask to reconfirm

Feature	Input	Processing	Output	Validation	Error Handling
Order tracking	order id	fetch order + ordered tracking-history	current status + full timeline	order must belong to requesting customer	403 if not owner, 404 if not found
Failed delivery handling	—	view failure remarks on tracking timeline	failure reason, attempted date	—	—
Rescheduling	order id, new delivery date	validate order is in <code>FAILED</code> state, date is in the future, write reschedule record, transition order to <code>RESCHEDULED</code> , trigger reassignment	updated order + new tracking entry	new date $\geq$ tomorrow, order must be owned by customer and in <code>FAILED</code> status	409 if order not in <code>FAILED</code> state, 422 if date invalid
Notifications	—	receives email on every status change and on reschedule confirmation	—	—	delivery failures of the notification itself must not block the status change
Order history	filters: status, date range	paginated query scoped to customer_id	list of orders with summary fields	page/limit bounds	400 on invalid pagination params

3.2 Delivery Agent

Feature	Details
Login	Same JWT flow as customer, role=AGENT. Agent accounts are created by Admin only (agents do not self-register).
Availability	Agent toggles <code>is_available</code> (true/false). Unavailable agents are excluded from auto-assignment.
Location/zone info	Agent has a <code>current_zone_id</code> (required) and optional <code>current_lat</code> / <code>current_lng</code> (updated via browser geolocation or manual entry) used for nearest-agent assignment.
Assigned orders	List of orders currently assigned to this agent, filterable by status.
Order details	Full order detail: addresses, package info, customer contact, charge, current status, history.
Status updates	Agent can move an order forward through: <code>ASSIGNED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED</code> , or <code>OUT_FOR_DELIVERY → FAILED</code> (failure can also be logged from <code>IN_TRANSIT</code> / <code>ASSIGNED</code> for undeliverable pickups). Each update requires the agent to be the currently assigned agent on that order.
Failed delivery	Agent marks <code>FAILED</code> with a mandatory reason (free text) and optional remarks; system notifies customer and unlocks rescheduling.
Rescheduled delivery handling	Once customer reschedules and admin/system reassigns an agent, the order reappears in the agent's (possibly new) assignment list with the new scheduled date.

3.3 Admin

Feature	Details
Login	Same JWT flow, role=ADMIN. First admin is created via seed script; admin can create additional admins/agents.
Dashboard	KPI summary: orders by status, orders by zone, active agents, failed-delivery count, today's deliveries.
Order management	View/search/filter all orders (status, zone, agent, order type, payment type, date range), view full detail + timeline for any order.
Customer management	View customer list, view a customer's order history, deactivate a customer account.
Agent management	Create/edit/deactivate agents, view agent workload (active order count), view agent availability & zone.
Zone management	Create/edit/delete zones (name, code, description).
Area-to-zone mapping	Create/edit/delete areas (name, pincode, city, state) and assign each area to exactly one zone.
Rate card configuration	CRUD on rate cards, one row per (order type × intra/inter-zone) combination: base price, base weight, price per additional kg, minimum charge.
COD surcharge configuration	CRUD on COD config per order type: surcharge type (flat/percentage), value, minimum charge.
Manual assignment	Select any available agent for an order regardless of algorithmic suggestion.
Automatic assignment	Trigger the nearest-available-agent algorithm for a single order or in bulk for all unassigned orders.
Status override	Force-set any order to any valid next status (or an explicit override with reason), always logged as an actor=ADMIN, actor_role=ADMIN tracking-history entry with <code>is_override=true</code> .
Order filtering	Filter list by status/zone/agent/order type/payment type/date range, combinable.
Monitoring	View notification delivery log (sent/failed), view unassigned/stuck orders.
Order creation on behalf of customer	Admin can create an order specifying an existing customer (search by email/phone) or creating a new customer inline; same pricing engine applies.

4. BUSINESS LOGIC

4.1 Zone Detection

IMPLEMENTATION DECISION — The source document requires zone detection but does not specify the geocoding mechanism. Full geocoding (lat/lng-based polygon zones) is unnecessary complexity for this scope and would require a paid maps API. Instead, use an **Area → Zone mapping table**: Admin pre-configures `Areas` (identified by pincode + city/state) and assigns each Area to exactly one `Zone`. Every address entered in the system must include a `pincode`, which is resolved against the `areas` table to derive its zone. This is deterministic, fully admin-configurable, requires no external API, and matches how real Indian courier rate cards are structured (pincode-to-zone mapping).

Rules:

Pickup zone = zone of the Area matching the pickup address's pincode.

Drop zone = zone of the Area matching the drop address's pincode.

Intra-zone: `pickup_zone_id == drop_zone_id`.

Inter-zone: `pickup_zone_id != drop_zone_id`.

Unmapped pincode: if a pincode has no matching Area, order creation/preview fails with `409`

`ZONE_NOT_CONFIGURED` and a message telling the customer the pincode is not yet serviceable. The system does **not** guess a zone.

Admin manages zones via CRUD; admin manages areas via CRUD, each area has a required `zone_id`.

Deleting a zone that has areas assigned is blocked (must reassign or delete areas first — enforce via FK `ON DELETE RESTRICT`)).

4.2 Volumetric Weight

Formula (from source, must be implemented exactly):

☐

$$\text{Volumetric Weight (kg)} = (\text{Length_cm} \times \text{Breadth_cm} \times \text{Height_cm}) / 5000$$

Units: L, B, H in centimeters; weight in kilograms.

Input validation: L, B, H, actual weight must each be strictly greater than 0 and numeric (decimals allowed, e.g., 12.5 cm). Reject 0, negative, non-numeric, or missing values with `422`.

Decimal handling: store dimensions as `DECIMAL(8,2)`, weights as `DECIMAL(8,3)`; round the computed volumetric weight to 3 decimal places using standard rounding (not truncation).

Billable weight = `MAX(actual_weight_kg, volumetric_weight_kg)`, also rounded to 3 decimals.

Worked examples (include these exact cases in seed/test data):

L	B	H	Actual Wt	Volumetric Wt	Billable Wt
30	20	15	2.0 kg	(30×20×15)/5000 = 1.8 kg	2.0 kg (actual wins)
50	40	30	4.0 kg	(50×40×30)/5000 = 12.0 kg	12.0 kg (volumetric wins)
25	25	20	2.5 kg	(25×25×20)/5000 = 2.5 kg	2.5 kg (tie — use either, document as tie-case)

4.3 Rate Calculation Engine

Full calculation pipeline, executed on both **preview** and **confirmation** (confirmation re-runs it to protect against stale rate cards):

Resolve **pickup zone** from pickup address pincode.

Resolve **drop zone** from drop address pincode.

Determine **zone relation**: `INTRA` or `INTER`.

Read **order type**: `B2B` or `B2C` (customer-supplied, required field).

Compute **volumetric weight**, then **billable weight** = MAX(actual, volumetric).

Look up the single active `RateCard` row matching `(order_type, zone_relation)`. If none exists → `409 RATE_NOT_CONFIGURED`.

Compute **base charge**:

```
if billable_weight_kg <= rate_card.base_weight_kg → base_charge = rate_card.base_price
else → base_charge = rate_card.base_price + (billable_weight_kg - rate_card.base_weight_kg) ×
rate_card.additional_price_per_kg
base_charge = MAX(base_charge, rate_card.min_charge)
```

Determine **payment type**: `PREPAID` or `COD`.

If `COD`: look up active `CODConfig` row for `order_type`. If none exists → `409 COD_NOT_CONFIGURED`.

If `surcharge_type = FLAT` → `cod_surcharge = codconfig.value`

If `surcharge_type = PERCENTAGE` → `cod_surcharge = MAX(base_charge × codconfig.value / 100, codconfig.min_charge)`

If `PREPAID` → `cod_surcharge = 0`

`total_charge = base_charge + cod_surcharge`, rounded to 2 decimals (currency).

Return the **full breakdown** (billable weight used, zone relation, base charge, COD surcharge, total) to the customer **before** they confirm the order. Nothing is charged/persisted at preview time.

IMPLEMENTATION DECISION — Rate card schema. The source specifies "intra and inter-zone rates separately for B2B and B2C" but not the pricing curve. A base-price + per-kg-beyond-base-weight model (as above) is used because it is the standard courier pricing pattern, fully admin-configurable via 4 numeric fields per rate card row, and avoids hardcoding weight slabs while still supporting non-linear-feeling pricing (a free allowance + marginal rate). Exactly one active rate card must exist per `(order_type, zone_relation)` pair — enforce with a unique constraint. No rates are ever hardcoded in code; all values are read from the `rate_cards` and `cod_configs` tables.

Missing rate handling: block order creation/preview with a clear `409` error naming the missing combination; admin dashboard surfaces a warning listing any `(order_type × zone_relation)` combination without an active rate card.

Invalid configuration handling: rate card CRUD validates `base_price ≥ 0`, `base_weight_kg > 0`, `additional_price_per_kg ≥ 0`, `min_charge ≥ 0`; reject invalid values at the API layer.

4.4 Auto-Assignment Logic

Agent availability model: `delivery_agents` table with `is_available` BOOLEAN, `current_zone_id`, optional `current_lat`/`current_lng`, `max_concurrent_orders` INT (default 5 — see decision below), and a derived `active_order_count` (count of orders where this agent is `current_agent_id` and status is in `ASSIGNED, PICKED_UP, IN_TRANSIT, OUT_FOR_DELIVERY`).

Agent location model: agents update `current_zone_id` (required, always present) and may optionally update `current_lat`/`current_lng` (via browser geolocation on the agent app, or manual entry) — `last_location_update_at` timestamp tracks freshness.

Selection algorithm (executed inside a single DB transaction with row-level locking to prevent races):
Candidate pool = agents where `role = AGENT`, `is_active = true`, `is_available = true`, and `active_order_count < max_concurrent_orders`.

If the order's **pickup zone** has ≥ 1 candidate with fresh (< 30 min old) `current_lat`/`current_lng`, AND the pickup address itself has a resolvable lat/lng (geocode the pincode centroid — store one lat/lng per Area) → compute **Haversine distance** between agent and pickup point for all such candidates; pick the minimum.

Zone fallback (used whenever step 2 has no usable coordinates, which will be the common case since GPS is optional): filter candidates to `current_zone_id == order.pickup_zone_id`. Among these, pick the one with the **lowest** `active_order_count` (load balancing); tie-break by **oldest** `last_assigned_at` (round robin — agents who haven't been assigned in the longest time go first).

No agent in pickup zone: broaden the candidate pool to *all* available agents system-wide, apply the same load-balancing + round-robin tie-break.

No agent available at all: order remains in `CREATED` status (unassigned). Admin dashboard lists it under "Unassigned Orders." Admin can retry auto-assignment or assign manually at any time. This is not an error state for the customer — the order still exists and is trackable.

Concurrency control: assignment is performed as `UPDATE delivery_agents SET active_order_count = active_order_count + 1 WHERE id = ? AND active_order_count < max_concurrent_orders` inside the same transaction as inserting the `order_assignments` row and updating `orders.current_agent_id` + `orders.status = ASSIGNED`. If the conditional update affects 0 rows (agent got filled by a concurrent request), retry selection from step 1.

Assignment history: every assignment (manual or auto) inserts a row into `order_assignments` (`order_id`, `agent_id`, `assigned_by_user_id`, `assignment_type`, `assigned_at`, `is_active`). Reassignment sets the previous row's `is_active = false` / `unassigned_at = now()` and inserts a new active row — full history is preserved, never overwritten.

Rescheduled deliveries: when a customer reschedules a `FAILED` order, the system re-runs the same selection algorithm (steps 1–6) to assign an agent for the new attempt; it is free to reassign the same agent or pick a different one — there is no "sticky agent" rule.

IMPLEMENTATION DECISION — `max_concurrent_orders` default of 5 and the 30-minute GPS freshness window are not specified in the source; they are reasonable operational defaults, both configurable by admin per agent / globally, not hardcoded into business logic.

4.5 Order Lifecycle

States: `CREATED → ASSIGNED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED` (terminal, success) or `→ FAILED → RESCHEDULED → ASSIGNED` (loop back into the pickup→delivery chain).

State	Meaning	Triggered by	Allowed previous states	Allowed next states	DB changes
CREATED	Order confirmed, charge locked in, awaiting agent	Customer/Admin confirming an order	— (initial)	ASSIGNED	insert order row
ASSIGNED	Agent assigned, not yet picked up	Auto-assignment or Admin manual assignment	CREATED, RESCHEDULED	PICKED_UP, FAILED	set <code>current_agent_id</code> , insert <code>order_assignments</code> row
PICKED_UP	Agent has collected the package	Agent (must be the assigned agent)	ASSIGNED	IN_TRANSIT, FAILED	update status, timestamp
IN_TRANSIT	Package en route	Agent	PICKED_UP	OUT_FOR_DELIVERY, FAILED	update status
OUT_FOR_DELIVERY	Agent is delivering today	Agent	IN_TRANSIT	DELIVERED, FAILED	update status
DELIVERED	Successfully delivered (terminal)	Agent	OUT_FOR_DELIVERY	— (terminal)	update status, <code>delivered_at</code>
FAILED	Delivery attempt failed	Agent (or Admin override)	ASSIGNED, PICKED_UP, IN_TRANSIT, OUT_FOR_DELIVERY	RESCHEDULED	update status, store failure reason/remarks
RESCHEDULED	Customer has requested a new delivery date	Customer	FAILED	ASSIGNED	insert <code>reschedules</code> row, set <code>scheduled_delivery_date</code>

Admin **status override** may force any transition (including otherwise-illegal ones) but is always logged with `actor_role = ADMIN`, `is_override = true`, and a mandatory remark explaining the override. Any transition attempt outside the "allowed next states" column (other than admin override) is rejected with `409 INVALID_STATUS_TRANSITION`.

4.6 Immutable Tracking History

Every single status change — including order creation, assignment, reassignment, agent status updates, reschedules, and admin overrides — inserts one row into `tracking_history`:

```
id, order_id, previous_status, new_status, changed_by_user_id, actor_role,
remarks (nullable), location_note (nullable), is_override (bool), created_at
```

Immutability enforcement (defense in depth):

API layer: no `PUT`/`PATCH`/`DELETE` endpoint is ever exposed for `tracking_history`. It is written only as a side effect inside the same transaction as a status change.

ORM layer: the model/repository for `tracking_history` exposes only a `create()` method — no `update`/`delete` methods are implemented at all, so there is nothing to call even by mistake.

Database layer: a dedicated, least-privilege application DB role is granted `INSERT, SELECT` only on `tracking_history` (no `UPDATE`/`DELETE` grants). Additionally add a Postgres trigger (`BEFORE UPDATE OR DELETE ON tracking_history`) that raises an exception, so even a raw SQL mistake or a compromised connection cannot rewrite history.

4.7 Failed Delivery Flow

```
Delivery attempt → Agent marks FAILED (reason required)
→ tracking_history entry written
→ customer notified by email (failure reason + "reschedule" link)
→ customer submits a new delivery date (must be ≥ tomorrow)
→ order → RESCHEDULED, reschedules row inserted, tracking_history entry written
→ system re-runs agent auto-assignment (Section 4.4) for the new date
→ order → ASSIGNED (new or same agent), tracking_history entry written, customer + agent notified
→ new delivery attempt proceeds through the normal PICKED_UP → ... chain
```

Rules:

Who can mark failure: the currently assigned Agent, or Admin (override, with remark).

Required data on failure: `reason` (enum: `CUSTOMER_UNAVAILABLE`, `ADDRESS_NOT_FOUND`, `CUSTOMER_REFUSED`, `OTHER`) + free-text `remarks` when `OTHER`.

Customer notification: immediate email including reason and a call-to-action to reschedule.

Reschedule validation: order must currently be `FAILED`; requester must be the order's owner (or Admin); new date must be a future date; a maximum of **3 reschedule attempts** per order is enforced (`409 MAX_RESCHEDULES_EXCEEDED` beyond that — admin can override).

Agent reassignment: automatic via Section 4.4 unless Admin manually reassigns.

Tracking history: both the `FAILED` and `RESCHEDULED` transitions (and the follow-on `ASSIGNED`) are separately logged rows, never merged.

5. DATABASE DESIGN

IMPLEMENTATION DECISION — PostgreSQL, accessed via Prisma ORM (TypeScript-native, type-safe migrations, works cleanly with Neon/Supabase free tiers used in Section 19). Table names are `snake_case`, plural.

5.1 Tables

users [id (uuid, pk)], [name], [email (unique, not null)], [phone (unique, not null)], [password_hash (not null)], [role (enum: CUSTOMER|AGENT|ADMIN, not null)], [is_active (bool, default true)], [created_at], [updated_at]

delivery_agents (1:1 extension of **users** where role=AGENT) [id (uuid, pk)], [user_id (fk users, unique, not null)], [is_available (bool, default false)], [current_zone_id (fk zones)], [current_lat (decimal, nullable)], [current_lng (decimal, nullable)], [last_location_update_at (timestamp, nullable)], [max_concurrent_orders (int, default 5)], [last_assigned_at (timestamp, nullable)], [created_at], [updated_at]

zones [id (uuid, pk)], [name (unique, not null)], [code (unique, not null)], [description (nullable)], [created_at], [updated_at]

areas [id (uuid, pk)], [name (not null)], [pincode (unique, not null, indexed)], [city], [state], [zone_id (fk zones, not null, on delete restrict)], [centroid_lat (decimal, nullable)], [centroid_lng (decimal, nullable)], [created_at], [updated_at]

addresses [id (uuid, pk)], [contact_name (not null)], [contact_phone (not null)], [line1 (not null)], [line2 (nullable)], [city], [state], [pincode (not null, indexed)], [area_id (fk areas, nullable – populated at resolution time)], [created_by_user_id (fk users)], [created_at]

rate_cards [id (uuid, pk)], [order_type (enum: B2B|B2C, not null)], [zone_relation (enum: INTRA|INTER, not null)], [base_price (decimal, not null)], [base_weight_kg (decimal, not null)], [additional_price_per_kg (decimal, not null)], [min_charge (decimal, not null)], [is_active (bool, default true)], [created_at], [updated_at] [UNIQUE (order_type, zone_relation) WHERE is_active = true]

cod_configs [id (uuid, pk)], [order_type (enum: B2B|B2C, not null)], [surcharge_type (enum: FLAT|PERCENTAGE, not null)], [value (decimal, not null)], [min_charge (decimal, default 0)], [is_active (bool, default true)], [created_at], [updated_at] [UNIQUE (order_type) WHERE is_active = true]

orders [id (uuid, pk)], [order_number (unique, not null, human-readable e.g. LMD-2026-000123)], [customer_id (fk users, not null)], [created_by_user_id (fk users, not null – customer or admin)], [pickup_address_id (fk addresses, not null)], [drop_address_id (fk addresses, not null)], [order_type (enum: B2B|B2C)], [payment_type (enum: PREPAID|COD)], [length_cm (decimal)], [breadth_cm (decimal)], [height_cm (decimal)], [actual_weight_kg (decimal)], [volumetric_weight_kg (decimal)], [billable_weight_kg (decimal)], [pickup_zone_id (fk zones)], [drop_zone_id (fk zones)], [zone_relation (enum: INTRA|INTER)], [base_charge (decimal)], [cod_surcharge (decimal)], [total_charge (decimal)], [status (enum, not null, default CREATED)], [scheduled_delivery_date (date, nullable)], [current_agent_id (fk delivery_agents, nullable)], [delivered_at (timestamp, nullable)], [reschedule_count (int, default 0)], [created_at], [updated_at]

order_assignments [id (uuid, pk)], [order_id (fk orders, not null)], [agent_id (fk delivery_agents, not null)], [assigned_by_user_id (fk users, not null)], [assignment_type (enum: MANUAL|AUTO)], [assigned_at (timestamp)], [unassigned_at (timestamp, nullable)], [is_active (bool, default true)]

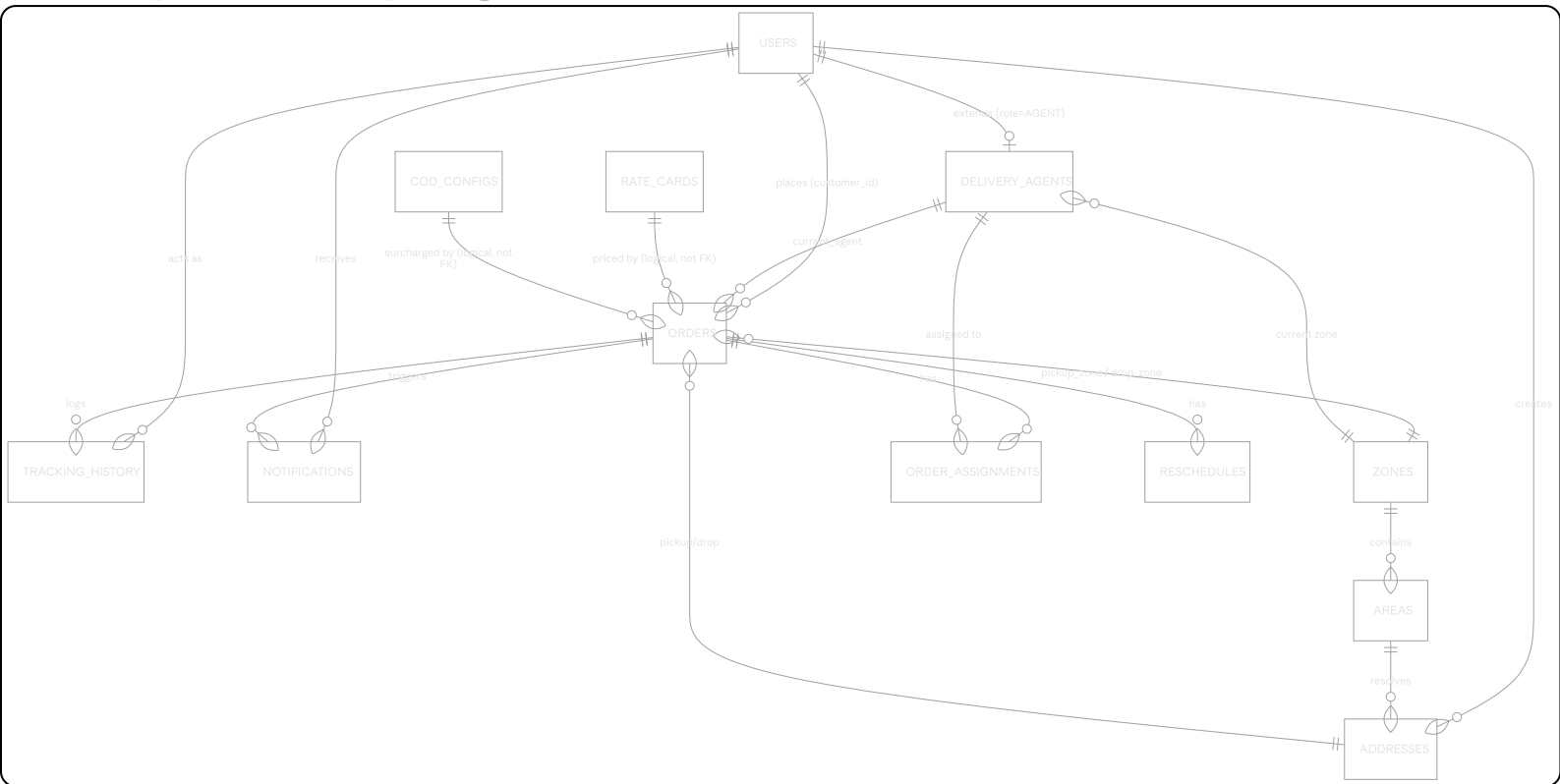
tracking_history (insert-only, see Section 4.6) [id (uuid, pk)], [order_id (fk orders, not null)], [previous_status (enum, nullable)], [new_status (enum, not null)], [changed_by_user_id (fk users, not null)], [actor_role (enum)], [remarks (nullable)], [location_note (nullable)], [is_override (bool, default false)], [created_at (not null)] Index on [(order_id, created_at)].

reschedules [id (uuid, pk)], [order_id (fk orders, not null)], [requested_by_user_id (fk users, not null)], [previous_scheduled_date (date, nullable)], [new_scheduled_date (date, not null)], [failure_reason_snapshot (enum, nullable)], [notes (nullable)], [created_at]

notifications [id (uuid, pk)], [user_id (fk users, not null)], [order_id (fk orders, nullable)], [channel (enum: EMAIL|SMS)], [event_type (text)], [recipient (text)], [subject (nullable)], [message (text)], [status (enum:

PENDING|SENT|FAILED)], provider_response (nullable), retry_count (int, default 0), created_at, sent_at (nullable)]

5.2 Entity Relationship Diagram



6. API ARCHITECTURE

Base path: [/api/v1]. JSON in/out. Consistent envelope:

Success: { "data": ..., "meta": {...optional pagination...} }

Error: { "error": { "code": "STRING_CODE", "message": "human readable", "fields": {...optional} } }

6.1 Authentication

Method	Endpoint	Purpose	Role	Auth
POST	/auth/register	Customer self-registration	Public	none
POST	/auth/login	Login, returns access + refresh token	Public	none
POST	/auth/refresh	Rotate access token using refresh cookie	Public	refresh cookie
POST	/auth/logout	Invalidate refresh token	Any	access token
GET	/auth/me	Current user profile	Any	access token

6.2 Customer

Method	Endpoint	Purpose	Role
POST	<code>/orders/quote</code>	Preview charge (Section 4.3), no persistence	CUSTOMER, ADMIN
POST	<code>/orders</code>	Create/confirm an order	CUSTOMER, ADMIN
GET	<code>/orders</code>	List own orders (filters: status, date range, pagination)	CUSTOMER
GET	<code>/orders/:id</code>	Order detail	CUSTOMER (own), ADMIN
GET	<code>/orders/:id/tracking</code>	Full immutable tracking timeline	CUSTOMER (own), AGENT (assigned), ADMIN
POST	<code>/orders/:id/reschedule</code>	Submit new delivery date after FAILED	CUSTOMER (own)

6.3 Agent

Method	Endpoint	Purpose	Role
PATCH	<code>/agents/me/availability</code>	Toggle <code>is_available</code>	AGENT
PATCH	<code>/agents/me/location</code>	Update zone / lat / lng	AGENT
GET	<code>/agents/me/orders</code>	Assigned orders (filter by status)	AGENT
PATCH	<code>/orders/:id/status</code>	Advance order status (PICKED_UP/IN_TRANSIT/OUT_FOR_DELIVERY/DELIVERED/FAILED)	AGENT (must be assigned to the order)



6.4 Admin

Method	Endpoint	Purpose	Role
GET	<code>/admin/orders</code>	All orders, filterable (status/zone/agent/order type/payment type/date)	ADMIN
PATCH	<code>/admin/orders/:id/status</code>	Override status (with mandatory remark)	ADMIN
POST	<code>/admin/orders/:id/assign</code>	Manual agent assignment	ADMIN
POST	<code>/admin/orders/:id/auto-assign</code>	Trigger auto-assignment for one order	ADMIN
POST	<code>/admin/orders/auto-assign-all</code>	Bulk auto-assign all unassigned orders	ADMIN
GET	<code>/admin/customers</code>	List/search customers	ADMIN
GET	<code>/admin/customers/:id/orders</code>	A customer's order history	ADMIN
GET	<code>/admin/agents</code>	List agents + workload	ADMIN
POST	<code>/admin/agents</code>	Create an agent account	ADMIN
PATCH	<code>/admin/agents/:id</code>	Edit/deactivate agent	ADMIN
GET	<code>/admin/zones</code>	List zones	ADMIN
POST	<code>/admin/zones</code>	Create zone	ADMIN
PATCH	<code>/admin/zones/:id</code>	Edit zone	ADMIN
DELETE	<code>/admin/zones/:id</code>	Delete zone (blocked if areas exist)	ADMIN
GET	<code>/admin/areas</code>	List areas	ADMIN
POST	<code>/admin/areas</code>	Create area (assign to zone)	ADMIN
PATCH	<code>/admin/areas/:id</code>	Edit area / reassign zone	ADMIN
DELETE	<code>/admin/areas/:id</code>	Delete area	ADMIN
GET	<code>/admin/rate-cards</code>	List rate cards	ADMIN
POST	<code>/admin/rate-cards</code>	Create rate card	ADMIN
PATCH	<code>/admin/rate-cards/:id</code>	Edit / deactivate rate card	ADMIN
GET	<code>/admin/cod-configs</code>	List COD configs	ADMIN

Method	Endpoint	Purpose	Role
POST	<code>/admin/cod-configs</code>	Create COD config	ADMIN
PATCH	<code>/admin/cod-configs/:id</code>	Edit / deactivate COD config	ADMIN
GET	<code>/admin/dashboard</code>	KPI summary	ADMIN
GET	<code>/admin/notifications</code>	Notification log	ADMIN

Every mutating endpoint documents: request body schema, validation rules, business logic invoked, success response shape, and every possible error code with its HTTP status. Every list endpoint supports `page`, `limit`, and relevant filter query params, returning `meta: { page, limit, total, totalPages }`.

7. AUTHENTICATION & AUTHORIZATION

IMPLEMENTATION DECISION — JWT-based stateless auth (no server-side session store needed), since it deploys cleanly on serverless/Render/Railway free tiers.

Password hashing: bcrypt, cost factor 12.

Tokens: short-lived **access token** (JWT, 15 min expiry, `Authorization: Bearer` header) + long-lived **refresh token** (JWT, 7 days, stored as an httpOnly, secure, sameSite=strict cookie, rotated on each refresh, invalidated on logout via a `refresh_tokens` denylist table keyed by token id).

Role-based middleware: `requireAuth` (validates access token) → `requireRole(...roles)` (checks `req.user.role` against an allow-list) applied per-route as shown in Section 6.

Resource ownership checks: customer-facing routes additionally verify `order.customer_id === req.user.id` (or admin); agent routes verify `order.current_agent_id === req.user.agentId`.

Unauthorized (no/invalid token): `401 { error: { code: "UNAUTHORIZED" } }`.

Forbidden (wrong role / not the owner): `403 { error: { code: "FORBIDDEN" } }`.

Password validation: min 8 chars, at least 1 letter and 1 number; never returned/logged in plaintext.

Security considerations: no role or user id is ever trusted from the request body — always derived from the verified JWT; agent/admin accounts cannot be created through the public register endpoint (agents/admins are created only by an existing Admin).

8. FRONTEND ARCHITECTURE

IMPLEMENTATION DECISION — No framework was specified in the source. Use **React 18 + TypeScript + Vite, Tailwind CSS** for styling, **React Router v6** for routing, **TanStack Query** for server-state/API caching, and **React Context** for lightweight global state (auth user, toasts). This stack is fast to build, free to host on Vercel, and keeps state management simple without introducing Redux for a project of this size.

Component architecture: feature-folder structure (`features/orders`, `features/agents`, `features/admin/zones`, etc.), each with `components/`, `hooks/`, `api.ts`. Shared `components/ui` for buttons, inputs, tables, modals, toasts.

Routing: `/login`, `/register`, `/dashboard` (role-aware redirect), `/orders/new`, `/orders/:id`, `/orders` (history), `/agent/*`, `/admin/*` — each `/admin/*` and `/agent/*` route wrapped in a `<ProtectedRoute role="ADMIN|AGENT">`.

State management: TanStack Query owns all server data (orders, zones, rate cards, etc.) with cache invalidation on mutations; Context holds only `currentUser` + `accessToken`.

API communication: a single `apiClient` (fetch/axios wrapper) attaches the bearer token, handles 401 by attempting a silent refresh once, then redirects to `/login` on repeated failure.

Protected routes: unauthenticated users are redirected to `/login`; authenticated users hitting a route outside their role are redirected to their own dashboard with a toast.

Error handling: every mutation surfaces API error messages via toast; forms show field-level errors from the `fields` map in the error envelope.

Loading states: skeleton loaders for lists/detail pages; disabled + spinner state on submit buttons.

Form validation: client-side validation mirrors backend rules (via a shared `zod` schema imported by both frontend and backend where practical) so users get instant feedback before hitting the API.

Responsive design: mobile-first Tailwind layout; agent app in particular must be fully usable on a phone screen since agents work in the field.

9. UI/UX SPECIFICATION

9.1 Customer UI

Screen	Purpose	Key elements
Landing / Login / Register	Auth entry	Email/password fields, role-aware redirect after login, link between login/register
Dashboard	Quick summary + shortcuts	Active orders count, "Create Order" CTA, recent orders list
Create Order	Enter order details	Pickup/drop address forms (with pincode), L/B/H, actual weight, order type, payment type toggle, live "Get Quote" button
Charge Preview	Show computed charge before commit	Billable weight used, zone relation, base charge, COD surcharge (if COD), total, "Confirm Order" / "Edit" buttons
Order Confirmation	Post-confirm success state	Order number, status=Created, "Track Order" link
Order Details	Full single-order view	All order fields, current status badge, assigned agent (if any), reschedule button when FAILED
Tracking Timeline	Visual history	Vertical stepper of every tracking_history row with timestamp + actor + remarks
Reschedule Screen	Pick new delivery date	Date picker (min = tomorrow), reason display from failed attempt, submit button
Order History	Browse past orders	Table/list with status filter, date range, pagination, click-through to details

9.2 Agent UI

Screen	Purpose	Key elements
Dashboard	Today's work at a glance	Assigned-today count, availability toggle prominent at top
Availability	Toggle online/offline, set current zone	Toggle switch, zone dropdown, "use my location" button (optional GPS)
Assigned Deliveries	List of assigned orders	Status filter tabs (Assigned/Picked Up/In Transit/Out for Delivery), sorted by scheduled date
Delivery Details	Full order info for the agent	Customer contact, pickup + drop address, package details, COD flag prominent
Status Update Controls	Advance the order	Buttons for the single valid next status (e.g., only "Mark Picked Up" shown while ASSIGNED), "Mark Failed" always available with reason dropdown
Failed Delivery Form	Capture failure	Reason dropdown (enum), remarks textarea (required if "Other"), submit

9.3 Admin UI

Screen	Purpose	Key elements
Dashboard	Operational overview	KPI cards (orders by status, unassigned count, failed count today), charts optional
Order Management	Browse/search all orders	Filter bar (status/zone/agent/order type/payment/date), table with pagination, row click → detail
Order Detail / Override	Deep dive + control	Full timeline, "Assign Agent" (manual dropdown or "Auto-assign" button), "Override Status" (dropdown + mandatory remark)
Zone Management	CRUD zones	Table + create/edit modal
Area Management	CRUD areas, map to zone	Table with pincode/city/zone columns + create/edit modal with zone dropdown
Rate Card Management	CRUD rate cards	Table grouped by order type, create/edit modal (base price, base weight, per-kg rate, min charge)
COD Configuration	CRUD COD surcharge rules	Table by order type, create/edit modal (flat vs percentage, value, min charge)
Agent Management	CRUD agents	Table with availability + workload columns, create/edit modal
Assignment Interface	Bulk view of unassigned orders	List with per-row "Assign" (manual) / "Auto-assign" buttons, "Auto-assign All" button

Every screen: defines empty states ("No orders yet — create your first order"), error states (inline banner with retry), and success states (toast + optimistic or refetched update). All admin tables must be production-quality: sortable columns, sticky headers, responsive collapse to cards on small screens.

10. NOTIFICATION SYSTEM

IMPLEMENTATION DECISION — Provider abstraction with two channels behind a common interface

`NotificationProvider.send(event)`. **Email:** Nodemailer against a free-tier SMTP provider (e.g., Resend or Gmail SMTP app-password) — fully functional, not mocked. **SMS:** the source requires "email and SMS integration... any free tier service" but does not mandate a specific provider; implement the same `send()` interface against **Twilio's free trial** and, since trial SMS requires verified numbers and is easy to exhaust, also ship a **console/log-based mock SMS provider** selected via an `SMS_PROVIDER=twilio|mock` env var so the app is always fully demoable even without live Twilio credentials. This satisfies "SMS integration works or has a documented free-tier implementation" from the Definition of Done.

Events that trigger notifications (email always; SMS on the terminal/urgent ones — order created, out for delivery, delivered, failed): Order created, agent assigned, picked up, in transit, out for delivery, delivered, failed (+ reason), reschedule confirmed.

Recipient: the order's customer (email + phone from their user record). Agent assignment also notifies the agent by email.

Provider abstraction: `services/notifications/provider.interface.ts` with `EmailProvider` and `SmsProvider` implementations, selected by env var, so swapping providers never touches business logic.

Retry behavior: on send failure, retry up to 3 times with exponential backoff (1s/5s/25s) via a lightweight in-process queue (or a simple `notifications` table poller) — never block the HTTP request that triggered the status change.

Failure handling: after 3 failed attempts, mark `notifications.status = FAILED`, keep `provider_response` for debugging; the order's status change itself is never rolled back due to a notification failure.

Notification logging: every attempt (success or failure) is a row in `notifications` (Section 5), visible to Admin at `/admin/notifications`.

11. PROJECT FOLDER STRUCTURE



astmile-delivery-tracker/

└─ backend/

└─ src/

└─└─ config/ # env loading, constants

└─└─ models/ (prisma schema lives in prisma/schema.prisma)

└─└─ modules/

└─└─└─ auth/ # controller, service, routes, validators

└─└─└─ orders/ # includes rate-calc-engine.ts, zone-detection.ts

└─└─└─ assignment/ # auto-assignment.service.ts

└─└─└─ agents/

└─└─└─ admin/ # zones, areas, rate-cards, cod-configs

└─└─└─ tracking/ # insert-only repository

└─└─└─ notifications/ # provider.interface.ts, email/, sms/

└─└─ middleware/ # auth.ts, requireRole.ts, errorHandler.ts, rateLimit.ts

└─└─ utils/

└─└─ jobs/ # notification retry worker

└─└─└─ app.ts / server.ts

└─ prisma/

└─└─ schema.prisma

└─└─ migrations/

└─└─└─ seed.ts

└─ tests/

└─└─ unit/

└─└─ integration/

└─└─└─ api/

└─ .env.example

└─└─ package.json

└─ frontend/

└─ src/

└─└─ features/

└─└─└─ auth/

└─└─└─ orders/

└─└─└─ agent/

└─└─└─ admin/

└─└─ components/ui/

└─└─ lib/apiClient.ts

└─└─ context/AuthContext.tsx

└─└─ routes/

└─└─└─ main.tsx

└─ .env.example

└─└─ package.json

└─ docs/

└─└─ API.md

└─└─ DB_SCHEMA.md

- └─ RATE_CALCULATION.md
- └─ SYSTEM_DESIGN.md
- ─ README.md
- ─ docker-compose.yml (optional local Postgres for dev)

12. ENVIRONMENT VARIABLES

backend/.env.example



```
# Server
PORT=4000
NODE_ENV=development
FRONTEND_URL=http://localhost:5173

# Database
DATABASE_URL=postgresql://user:password@host:5432/lastmile_db

# Auth
JWT_ACCESS_SECRET=replace_me
JWT_REFRESH_SECRET=replace_me
JWT_ACCESS_EXPIRY=15m
JWT_REFRESH_EXPIRY=7d

# Email
EMAIL_PROVIDER=smtp
SMTP_HOST=smtp.resend.com
SMTP_PORT=587
SMTP_USER=replace_me
SMTP_PASS=replace_me
EMAIL_FROM=no-reply@lastmiletracker.dev

# SMS
SMS_PROVIDER=mock      # mock | twilio
TWILIO_ACCOUNT_SID=replace_me
TWILIO_AUTH_TOKEN=replace_me
TWILIO_FROM_NUMBER=replace_me

# Assignment
AGENT_LOCATION_FRESHNESS_MINUTES=30
DEFAULT_MAX_CONCURRENT_ORDERS=5

# Rate limiting
LOGIN_RATE_LIMIT_MAX=5
LOGIN_RATE_LIMIT_WINDOW_MINUTES=15
```

frontend/.env.example

```

VITE_API_BASE_URL=http://localhost:4000/api/v1
```

Never commit a real `.env` file; `.gitignore` must exclude it. `.env.example` is committed with placeholder values only.

13. VALIDATION & ERROR HANDLING

Validate at both the API boundary (server, authoritative) and the frontend forms (UX only).

Domain	Rules
Registration	valid email format, unique email/phone, password policy (Section 7)
Login	required fields, no user enumeration in error message ("invalid credentials" generic)
Addresses	pincode required + numeric, resolvable to a configured Area (else <code>ZONE_NOT_CONFIGURED</code>)
Dimensions	L/B/H each <code>> 0</code> , numeric, max 2 decimal places
Weight	actual weight <code>> 0</code> , numeric, max 3 decimal places
Order type	must be <code>B2B</code> or <code>B2C</code>
Payment type	must be <code>PREPAID</code> or <code>COD</code>
Zones	name/code required + unique; delete blocked if areas reference it
Rate cards	numeric fields <code>≥ 0</code> (<code>(base_weight_kg > 0)</code>), unique active <code>(order_type, zone_relation)</code>
COD	numeric <code>value ≥ 0</code> ; unique active per <code>order_type</code>
Agent assignment	agent must be <code>is_available</code> , under <code>max_concurrent_orders</code> , and (for manual) must exist and be active
Status transitions	must match the allowed-next-states table (Section 4.5) unless admin override
Rescheduling	order must be <code>FAILED</code> , new date $\geq$ tomorrow, reschedule count <code>< 3</code>

Consistent error envelope + status codes:

- `400` malformed request / missing required field
- `401` unauthenticated
- `403` forbidden (wrong role / not resource owner)
- `404` resource not found
- `409` business-rule conflict (`ZONE_NOT_CONFIGURED`, `RATE_NOT_CONFIGURED`, `INVALID_STATUS_TRANSITION`, `MAX_RESCHEDULES_EXCEEDED`, duplicate email, etc.)
- `422` validation failure with field-level `fields` map
- `429` rate limited
- `500` unhandled server error (never leaks stack traces in production responses; logged server-side)

14. SECURITY REQUIREMENTS

Passwords hashed with bcrypt (cost 12); never logged or returned.

JWT auth with short-lived access tokens + rotated refresh tokens (Section 7).

Role-based authorization enforced on every non-public route; ownership checks on customer/agent-scoped resources.

All input validated and sanitized server-side (schema validation library, e.g., `zod`) before touching the database.

SQL injection prevention: exclusively via Prisma's parameterized queries — no raw string-concatenated SQL.

CORS: restrict to `FRONTEND_URL` origin only in production; credentials enabled for the refresh-token cookie.

Rate limiting: on `/auth/login` and `/auth/register` (Section 12 config) to blunt brute-force attempts.

Secrets only via environment variables, never hardcoded, never committed.

Prevent unauthorized status changes: enforced by role middleware + "must be assigned agent" ownership check + the allowed-transition table.

Prevent modification of immutable tracking history: three-layer defense in Section 4.6.

Prevent duplicate assignments: conditional atomic update + unique-active-assignment invariant (Section 4.4 step 6).

Protect admin endpoints: every `/admin/*` route behind `requireRole('ADMIN')`; agent-management and rate-card endpoints double-checked with integration tests (Section 15).

15. TESTING STRATEGY

Unit Tests

Volumetric weight formula (including the tie-case example from Section 4.2).

Billable weight selection (actual wins / volumetric wins / tie).

Rate calculation engine: base charge under base weight, over base weight, at exact base weight boundary.

COD surcharge: flat, percentage, percentage-below-min-charge.

Zone determination: intra vs inter, unmapped pincode error path.

Agent assignment selection: GPS path, zone-fallback path, load-balancing tie-break, no-agent-available path.

Status transition validator: every allowed transition passes, every disallowed transition is rejected, admin override bypasses validator.

Integration Tests

Full order creation: address → zone resolution → rate lookup → persisted order → tracking_history row created.

Assignment: manual assignment updates `current_agent_id` + creates `order_assignments` row; auto-assignment end-to-end against seeded agents.

Status updates: agent walks an order through the full happy path, verifying a tracking_history row per step.

Failed delivery → notification row created → reschedule → reassignment end-to-end.

Notification retry: force a provider failure, assert 3 retries then `FAILED` status logged, and that the order status itself is unaffected.

API Tests

One test per endpoint in Section 6 covering: happy path, validation failure, and (where applicable) the relevant 409 business error.

Authorization Tests

Every `/admin/*` endpoint rejected for CUSTOMER and AGENT tokens (403).

Every `/agents/me/*` and status-update endpoint rejected for a non-assigned agent (403).

Every customer order/tracking endpoint rejected for a different customer's order (403/404).

All protected endpoints reject missing/expired/invalid tokens (401).

Edge Cases (must have explicit tests)

Missing zone (unmapped pincode on pickup and on drop, independently).

No available agent anywhere → order stays `CREATED`, correctly surfaced on admin "unassigned" list.

Equal actual/volumetric weight (tie-case).

Zero/negative dimensions and weight → `422`.

Invalid weight (non-numeric, absurdly large) → `422`.

Invalid/missing rate configuration for a given `(order_type, zone_relation)` → `409 RATE_NOT_CONFIGURED`.

Duplicate status transition (e.g., calling "mark delivered" twice) → `409`.

Invalid/out-of-order status transition (e.g., `CREATED → DELIVERED` directly) → `409`.

Failed delivery with no reschedule submitted (order correctly stays `FAILED`, visible to customer with reschedule CTA).

Unauthorized admin operation attempted by a customer/agent token.

Multiple agents with equal distance/equal load → deterministic tie-break (round robin) verified by test.

Agent becomes unavailable (or hits `max_concurrent_orders`) mid-assignment race → conditional update retry verified with a concurrency test (two simultaneous assignment requests against one remaining slot).

16. ACCEPTANCE CRITERIA



text

AC-001:

Given a customer submits pickup pincode 600001 (Zone A) and drop pincode 600020 (Zone A)

When the order is priced

Then the zone_relation is INTRA and the INTRA rate card for the order's type is used.

AC-002:

Given a customer submits pickup pincode 600001 (Zone A) and drop pincode 400001 (Zone B)

When the order is priced

Then the zone_relation is INTER and the INTER rate card for the order's type is used.

AC-003:

Given package dimensions 50x40x30 cm and actual weight 4.0 kg

When volumetric weight is calculated

Then volumetric_weight_kg = 12.0 and billable_weight_kg = 12.0 (volumetric wins).

AC-004:

Given package dimensions 30x20x15 cm and actual weight 2.0 kg

When volumetric weight is calculated

Then volumetric_weight_kg = 1.8 and billable_weight_kg = 2.0 (actual wins).

AC-005:

Given payment_type = COD and an active COD config with surcharge_type=PERCENTAGE, value=2, min_

When the order is priced with base_charge = 500

Then cod_surcharge = 20 (since 2% of 500 = 10, which is below the 20 minimum, so min_charge applies).

AC-006:

Given no rate card exists for (order_type=B2B, zone_relation=INTER)

When a customer requests a quote for a B2B inter-zone order

Then the API returns 409 RATE_NOT_CONFIGURED and no order is created.

AC-007:

Given a pincode not present in the areas table

When a customer attempts to create an order using that pincode

Then the API returns 409 ZONE_NOT_CONFIGURED and no order is created.

AC-008:

Given an order in status CREATED and no available agents in any zone

When admin triggers auto-assign

Then the order remains CREATED, is listed under admin's "Unassigned Orders", and no tracking_history row

AC-009:

Given two available agents in the pickup zone with active_order_count 1 and 3 respectively

When auto-assignment runs

Then the agent with active_order_count = 1 is selected.

(Antigravity must generate additional AC-### entries for every remaining endpoint and rule in this document following the same Given/When/Then format before considering the project complete.)

17. SAMPLE TEST DATA

Seed the following (no real personal data — use clearly fictional names/emails):

Admin: `admin@lastmiletracker.dev` / `Admin@1234`

Customers: `priya.b2c@example.com` (individual, places B2C orders), `acme.traders.b2b@example.com` (business account, places B2B orders) — both password `Customer@1234`

Agents: 4 agents across 2 zones — `agent.north1@example.com`, `agent.north2@example.com` (Zone: North), `agent.south1@example.com`, `agent.south2@example.com` (Zone: South) — password `Agent@1234`; vary `is_available` (one offline) and `active_order_count` to exercise load balancing

Zones: `North`, `South`

Areas: 2 pincodes mapped to North (e.g., 600001, 600020), 2 mapped to South (e.g., 400001, 400020), plus one deliberately **unmapped** pincode used only in tests (e.g., 999999) to exercise `ZONE_NOT_CONFIGURED`

Rate cards: all 4 combinations seeded — (B2C, INTRA), (B2C, INTER), (B2B, INTRA), (B2B, INTER) — with distinct base/per-kg values so tests can distinguish them

COD configs: one FLAT for B2C, one PERCENTAGE for B2B

Orders (seed a representative set covering every dimension called out by the source):

B2C, intra-zone, Prepaid, actual-weight-wins, status `DELIVERED`

B2C, inter-zone, COD, volumetric-weight-wins, status `IN_TRANSIT`

B2B, intra-zone, Prepaid, status `OUT_FOR_DELIVERY`

B2B, inter-zone, COD, status `ASSIGNED`

B2C, intra-zone, Prepaid, status `FAILED` (with a `CUSTOMER_UNAVAILABLE` reason) — ready to demo rescheduling

B2C, intra-zone, Prepaid, status `RESCHEDULED` → `ASSIGNED` (demonstrates reassignment, assigned to a **different** agent than the original)

B2B, intra-zone, Prepaid, status `CREATED` (unassigned — no agent available, demonstrates the admin "Unassigned Orders" view)

18. SEEDING & DEMO FLOW

Implement `backend/prisma/seed.ts`, run via `npm run seed`, idempotent (safe to re-run — upsert by unique keys).

After `npm install && npm run migrate && npm run seed && npm run dev` (backend) and `npm install && npm run dev` (frontend), the app must be immediately usable end-to-end with the credentials from Section 17.

Document all demo credentials clearly and prominently in the README (Section 20), in a single table.

The seed data (Section 17) must, by itself, let a reviewer log in as each of the three roles and observe every stage of the lifecycle (created/unassigned, assigned, in transit, delivered, failed, rescheduled) without creating anything new — though creating a brand-new order end-to-end through the UI must also work flawlessly.

19. DEPLOYMENT ARCHITECTURE

IMPLEMENTATION DECISION — the source lists Vercel/Render/Railway/similar as acceptable, without a specific mandate. Use: **Frontend → Vercel, Backend → Render** (free web service, auto-deploy from `main`),

Database → Neon (free-tier serverless Postgres, connection pooling built in — plays well with Prisma).

Frontend deployment: Vercel project rooted at `/frontend`; build command `npm run build`; env var `VITE_API_BASE_URL` set to the deployed backend URL.

Backend deployment: Render web service rooted at `/backend`; build `npm install && npm run build`; start `npm run start`; all Section 12 env vars set in Render's dashboard (never committed).

Database deployment: Neon project; `DATABASE_URL` (pooled connection string) set as a Render env var.

CORS: backend `FRONTEND_URL` env var set to the live Vercel URL so CORS allows exactly that origin in production.

Production build: frontend built as static assets served by Vercel's CDN; backend compiled TypeScript → `dist/` run under Node.

Migration process: `npx prisma migrate deploy` run as part of Render's build/release step (not `migrate dev`, which is dev-only).

Seed process: run once manually after first deploy (`npm run seed` against the production `DATABASE_URL`) to populate demo data — document this exact command in the README.

Health check: `GET /api/v1/health` returning `{ status: "ok", db: "connected" }`, wired into Render's health-check config.

API URL configuration: the frontend never hardcodes an API origin — always reads `VITE_API_BASE_URL` at build time.

20. README REQUIREMENTS

The repository root `README.md` must include, in this order: project overview; feature list; architecture diagram/summary; tech stack table; prerequisites (Node version, Postgres); local installation steps; environment variable setup (link to `.env.example`); database setup + migration commands; seed data command + demo credentials table (Section 17); "running locally" (both frontend and backend, in one combined set of steps); API documentation (link to `docs/API.md`, generated from Section 6); authentication explanation (Section 7 summarized); rate calculation logic explanation (Section 4.2–4.3 summarized, link to `docs/RATE_CALCULATION.md`); agent assignment logic explanation (Section 4.4 summarized); order lifecycle diagram (Section 4.5 state table); failed delivery flow (Section 4.7 diagram); testing instructions (`npm test` per package, coverage command); deployment instructions (Section 19, with the actual live URL once deployed); demo credentials repeated at the bottom for quick access.

21. SYSTEM DESIGN WRITE-UP

Produce `docs/SYSTEM_DESIGN.md`, **maximum 800 words**, covering exactly these six areas from the source requirements, each as its own short subsection:

Rate calculation engine — the pipeline in Section 4.3, why base+per-kg was chosen over hardcoded slabs, how it stays fully admin-configurable.

Zone detection approach — the Area→Zone pincode-mapping decision from Section 4.1 and why full geocoding was avoided.

Auto-assignment logic — the GPS-first/zone-fallback/load-balancing algorithm from Section 4.4 and its concurrency safety.

Failed delivery handling — the state machine loop from Section 4.7.

Database architecture — the core entities from Section 5 and why `tracking_history` is append-only.

Status tracking — how immutability is enforced at three layers (Section 4.6).

Write this only after the implementation is complete, so it accurately reflects what was actually built, not just what was planned. Keep it under 800 words — this is a hard limit from the source requirements.

22. CODE QUALITY REQUIREMENTS

Clean, layered architecture: `routes → controllers → services (business logic) → repositories (data access)`.

Controllers never contain business logic; services never touch `req/res`.

Meaningful, descriptive names throughout (no `data2`, `tmp`, `foo`).

No duplicated logic — e.g., the rate engine and zone-detection logic each live in exactly one module, imported everywhere they're needed (including by tests).

Reusable frontend components for repeated UI patterns (status badges, data tables, form fields).

Business logic strictly separated from controllers and UI components.

All configuration via environment variables — nothing environment-specific hardcoded.

Comments only where the "why" isn't obvious from the code itself (e.g., explain the tie-break rule, not `// increment i`).

Consistent formatting via Prettier + ESLint (backend and frontend), enforced in CI if CI is set up.

Errors always handled — no empty `catch` blocks, no unhandled promise rejections.

No unnecessary dependencies — every package added must be justified by an actual requirement in this document.

No hardcoded business configuration (rates, COD surcharges, zones) anywhere in code — always read from the database.

Frontend and backend remain fully independent deployable units, communicating only over the documented REST API.

23. DEVELOPMENT STRATEGY

Build in the following phases. **After each phase**, verify the implementation, run the relevant tests for that phase, fix any errors, confirm integration with all previous phases, and only then proceed.

Phase 1 — Project setup & architecture: repo scaffolding, folder structure (Section 11), linting/formatting config, base Express app + base React app wired together with a health-check round trip.

Phase 2 — Database & migrations: Prisma schema (Section 5), initial migration, seed script skeleton.

Phase 3 — Authentication & RBAC: register/login/refresh/logout, JWT middleware, role middleware, protected route smoke tests.

Phase 4 — Zone & rate-card management: zones/areas CRUD, rate-card CRUD, COD-config CRUD, admin UI for all of the above.

Phase 5 — Rate calculation engine: volumetric/billable weight, zone detection, full pricing pipeline (Section 4.3) with unit tests from Section 15 passing.

Phase 6 — Order creation: quote endpoint, order confirmation endpoint, customer "Create Order" + "Charge Preview" UI.

Phase 7 — Agent assignment: agent model, availability/location endpoints, manual assignment, auto-assignment algorithm (Section 4.4) with concurrency tests passing.

Phase 8 — Order lifecycle & tracking: status-update endpoint with transition validation, immutable tracking_history writes + DB trigger, tracking timeline UI.

Phase 9 — Failed delivery & rescheduling: failure capture, reschedule endpoint + validation, reassignment trigger, related UI screens.

Phase 10 — Notifications: provider abstraction, email provider wired for real, SMS mock + Twilio path, retry worker, admin notification log.

Phase 11 — Frontend dashboards: complete customer/agent/admin dashboards and remaining screens from Section 9.

Phase 12 — Testing: fill out the full test suite from Section 15 across unit/integration/API/authorization/edge cases.

Phase 13 — Documentation: README, `docs/API.md`, `docs/DB_SCHEMA.md`, `docs/RATE_CALCULATION.md`, `docs/SYSTEM_DESIGN.md` (Section 21).

Phase 14 — Deployment: deploy per Section 19, run production migration + seed, verify the live URL end-to-end against the Definition of Done (Section 25).

24. ANTIGRAVITY EXECUTION RULES

Rule 1 — Source of Truth: the requirements and specifications in this document are authoritative.

Rule 2 — No Hardcoding: business configuration (rates, COD surcharges, zone/area mappings) must always be database/admin-configurable — never hardcoded in application code.

Rule 3 — No Fake Functionality: never build a button, screen, or endpoint that appears functional but has no real backend behind it.

Rule 4 — End-to-End Integration: every frontend feature must call a real backend API — no mocked frontend data in the shipped product.

Rule 5 — Validation: validate every input on both frontend and backend.

Rule 6 — Security: never expose secrets, never bypass authorization checks, even temporarily "to test something."

Rule 7 — Auditability: every order status change must produce an immutable `tracking_history` record — no exceptions, including admin overrides.

Rule 8 — Testing: business-critical logic (rate engine, assignment, lifecycle transitions) must be tested before being considered complete.

Rule 9 — Error Handling: never leave an unhandled error or a broken user flow in the shipped product.

Rule 10 — Documentation: whenever an implementation decision changes during development, update the relevant documentation (README/docs) to match.

Rule 11 — Production Readiness: do not stop at a prototype — the project must be deployable and demonstrable end-to-end at a live URL.

25. DEFINITION OF DONE

- ☐ Customer registration works
- ☐ Customer login works
- ☐ Agent login works
- ☐ Admin login works
- ☐ RBAC works (every role blocked from every other role's endpoints)
- ☐ Zones work (CRUD)
- ☐ Areas map to zones (CRUD + pincode resolution)
- ☐ Rate cards are configurable (CRUD, no hardcoded values anywhere)
- ☐ COD surcharge is configurable (CRUD)
- ☐ Volumetric weight works (formula verified against worked examples)

- ☐ Billable weight works (MAX logic verified, including tie-case)
- ☐ B2B rates work
- ☐ B2C rates work
- ☐ Intra-zone rates work
- ☐ Inter-zone rates work
- ☐ Order charge is shown before confirmation (no persistence at preview time)
- ☐ Orders can be created (by customer and by admin-on-behalf-of-customer)
- ☐ Agents can be manually assigned
- ☐ Auto-assignment works (GPS path + zone-fallback path + load balancing)
- ☐ Agent availability works
- ☐ Order statuses work (full lifecycle, transition validation enforced)
- ☐ Tracking history is immutable (verified via all three enforcement layers)
- ☐ Failed delivery works (reason capture, customer notification)
- ☐ Rescheduling works (validation, max-attempts limit)
- ☐ Agent reassignment works (on reschedule)
- ☐ Email notifications work (real provider, not mocked)
- ☐ SMS integration works or has a documented free-tier/mock implementation
- ☐ Admin can filter orders (status/zone/agent/order type/payment type/date)
- ☐ Admin can override order status (with mandatory remark, logged)
- ☐ Tests pass (unit, integration, API, authorization, edge cases from Section 15)
- ☐ README is complete (Section 20)
- ☐ `.env.example` exists for both frontend and backend
- ☐ API documentation exists (`docs/API.md`)
- ☐ Database schema is documented (`docs/DB_SCHEMA.md` + ERD)
- ☐ Rate calculation is documented (`docs/RATE_CALCULATION.md`)
- ☐ System design write-up is complete and ≤ 800 words (`docs/SYSTEM_DESIGN.md`)
- ☐ Production deployment works (frontend, backend, database all live and connected)
- ☐ Hosted URL is available and functional end-to-end with seed demo credentials

Build the entire project against this specification without pausing to ask what to build next. Where you must make a judgment call not covered above, make the most production-sensible choice, implement it, and note it in the README under "Additional Implementation Decisions."